

Plan de Implementación

Sistema de Registro de Comprobantes — Marangatu / DNIT Paraguay

Versión: 0.1 (borrador inicial) Fecha: 2026-05-13

Este documento resume el plan de implementación acordado tras el análisis de la planilla modelo de Marangatu, los libros contables auxiliares de la SET y la Especificación Técnica para Importación (junio/2021). Refleja las decisiones tomadas con el usuario el 2026-05-13 sobre alcance, roles, despliegue y volumen.

1. Resumen ejecutivo

Aplicación web SaaS multi-tenant para asistir a estudios contables paraguayos en el registro de comprobantes (fotos, imágenes, PDFs). Usa la API de Gemini para extraer datos visualmente, los valida contra las reglas de la SET/DNIT y genera tres tipos de exportación:

- Archivo CSV/TXT en ZIP con el nombre exacto que exige el módulo de Importación de Marangatu (obligación 955 mensual o 956 anual).
- Planilla Excel modelo Marangatu (4 hojas: VENTAS, COMPRAS, INGRESOS, EGRESOS).
- Libros contables auxiliares según régimen del cliente: Libro IVA, Libro IVA+IRE, Libro IRP puro.

Gemini sólo extrae datos. La imputación tributaria (IVA / IRE / IRP-RSP / NO IMPUTA) la define el usuario en base al régimen del cliente. La aprobación humana es obligatoria antes de registrar.

2. Decisiones clave acordadas con el usuario

#	Decisión	Justificación
D-010	Multi-usuario con roles desde V1 (Admin + Operador)	Admin aprueba/exporta; Operador carga y deja pendientes.
D-011	SaaS multi-tenant en la nube	Permite que varios estudios usen la misma plataforma con aislamiento estricto.
D-012	Soporte simultáneo de IVA, IVA+IRE y IRP-RSP	Los clientes del estudio mezclan los tres regímenes.
D-013	Cola robusta con BullMQ + Redis desde V1	Volumen alto declarado (>1.000 comprobantes/mes).
D-014	Importar e-Kuatia (XML) sólo para libros internos; SET y libros de ZAP para electrónicos	El usuario igual quiere verlos con su propio ojo.

3. Stack técnico propuesto

Capa	Tecnología
Frontend	Next.js 15 (App Router) + React 19 + TypeScript + TailwindCSS + shadcn/ui
Visor PDF/imagen	pdf.js (react-pdf)
Backend / API	Next.js Route Handlers + servicios en TS
Auth	NextAuth (Auth.js) con credenciales + roles
ORM	Prisma
Base de datos	PostgreSQL (Supabase o Neon) con RLS por tenant

Capa	Tecnología
Cola de jobs	BullMQ + Redis (Upstash)
Storage de archivos	Cloudflare R2 (S3-compatible, sin egress)
IA	Gemini 2.5 (Pro o Flash) vía @google/genai — server-side
Excel	ExcelJS
Observabilidad	Sentry + Axiom/Logtail
Testing	Vitest (unit) + Playwright (E2E)

4. Plan de implementación por fases

Estimación total: **9–10 semanas** de trabajo enfocado.

Fase 0 — Cimientos SaaS (1.5 semanas)

- Setup repo + Next.js 15 + Tailwind + Prisma + Postgres (Supabase o Neon).
- Schema Prisma inicial: organizacion, usuario, cliente, comprobante, archivo, catálogos.
- Seed de Tablas 1 a 5 de la SET (junio/2021).
- NextAuth con credenciales + roles admin / operador.
- Onboarding: signup crea organización + primer admin.
- Bucket Cloudflare R2 + helper de URLs firmadas.
- Redis (Upstash) + BullMQ con worker placeholder.
- **Entregable:** SaaS vacío con signup/login multi-tenant.

Fase 1 — Gestión de clientes y carga (1 semana)

- CRUD de clientes con validación de RUC + cálculo de DV.
- Selector de cliente activo persistente en sesión.
- Upload con hash SHA-256, almacenamiento por organización y cliente, deduplicación.
- Visor PDF.js / imagen con zoom, rotación, multipágina.
- **Entregable:** se cargan comprobantes y se ven por cliente.

Fase 2 — Extracción Gemini + revisión (2 semanas)

- Cliente Gemini server-side con prompt estricto y JSON schema.
- Worker BullMQ de extracción con reintentos.
- Persistencia de campo_extraido (valor, confianza, status, bbox).
- Pantalla de revisión split: visor + panel de campos.
- Sistema de badges de confianza por campo y general.
- Edición con auditoría inmutable.
- Aprobación, rechazo, pendiente, re-extracción focalizada.
- **Entregable:** flujo end-to-end desde carga a registro.

Fase 3 — Validaciones DNIT completas (1 semana)

- Motor de validación con V-001 a V-022 y C-001 a C-007.
- Detección de duplicados por (cliente, RUC, timbrado, número, fecha).
- Bloqueos por confianza baja en campos críticos.
- **Entregable:** validación cumpliendo Especificación Técnica.

Fase 4 — Exportaciones (2 semanas)

- ZIP Marangatu con nombre <RUC>_REG_MMAAAA_XXXXX.zip (mensual, 955) o <RUC>_REG_AAAA_XXXXX.zip (anual, 956).

- Exclusión automática de comprobantes con origen E_KUATIA_XML.
- Paginación a 5.000 filas por archivo.
- Excel Planilla Marangatu (4 hojas, cabeceras fila 10).
- Excel Libro IVA / IVA+IRE.
- Excel Libro IRP puro.
- Importador de XML e-Kuatia (carga directa a REGISTRADO con origen E_KUATIA_XML).
- Registro de exportaciones con hash.
- **Entregable:** archivos listos para subir a Marangatu y libros consolidados.

Fase 5 — Historial, auditoría, dashboard (1 semana)

- Historial filtrable por estado, período, contraparte.
- Pantalla de auditoría con búsqueda.
- Dashboard del cliente activo con KPIs del mes.
- **Entregable:** observabilidad operativa para el contador.

Fase 6 — Hardening y despliegue productivo (1 semana)

- Backups automáticos (pg_dump + R2 versioning).
- Tests E2E con Playwright sobre comprobantes muestra.
- Aislamiento cross-tenant verificado en CI.
- Documentación de usuario.
- Despliegue productivo (Vercel/Fly + Supabase/Neon + Upstash + R2).
- **Entregable:** sistema en producción.

5. Backlog detallado (Fases 0 y 1)

Fase 0

- F0-1 — Init repo, estructura del proyecto.
- F0-2 — Next.js 15 + TS strict + Tailwind + shadcn/ui.
- F0-3 — Supabase o Neon: proyecto creado, variables de entorno.
- F0-4 — Prisma init y schema base con organizacion + usuario + cliente + archivo + catálogos.
- F0-5 — Migración inicial y RLS por organizacion_id.
- F0-6 — Seed con Tablas 1 a 5 SET (junio/2021).
- F0-7 — NextAuth (credenciales + Google opcional) con roles.
- F0-8 — Onboarding: signup crea organización + admin inicial.
- F0-9 — Layout base (header con selector de cliente, sidebar con menú).
- F0-10 — Página de configuración con campo API Key Gemini cifrada.
- F0-11 — Bucket R2 + util de URLs firmadas.
- F0-12 — Upstash Redis + BullMQ con worker placeholder.
- F0-13 — Pipeline CI mínimo (lint + typecheck + tests).

Fase 1

- F1-1 — Validador de RUC paraguayo + cálculo DV (módulo 11).
- F1-2 — UI de alta/edición/baja lógica de cliente.
- F1-3 — Listado de clientes filtrable.
- F1-4 — Selector global de cliente activo en sesión.
- F1-5 — Drag-and-drop de archivos con preview previo a subida.
- F1-6 — Endpoint de upload (multipart, validación MIME, hash SHA-256).
- F1-7 — Persistencia en R2 con prefijo org/<id>/clientes/...
- F1-8 — Detección de duplicado por hash con advertencia.
- F1-9 — Listado de comprobantes por cliente con estado.
- F1-10 — Página de detalle con visor PDF.js/imagen.
- F1-11 — Tests unitarios del validador RUC + tests de upload.

6. Criterios de aceptación globales

- Todo comprobante registrado conserva el archivo original con hash verificable.
- Ningún comprobante puede registrarse si viola las reglas V-001 a V-022 (ver 09_VALIDACIONES.md).
- El ZIP exportado debe poder cargarse en Marangatu sin errores (verificación cruzada contra el modelo del PDF SET).
- Las exportaciones Excel respetan exactamente las cabeceras de los modelos oficiales (fila 10 en planilla; fila 11 en libros).

- Toda modificación post-registro queda en auditoria_cambio con motivo.
- Comprobantes con origen E_KUATIA_XML nunca aparecen en el ZIP Marangatu.
- Pruebas E2E incluyen aislamiento entre organizaciones (acceso cruzado debe devolver 403).

7. Pendientes no bloqueantes para iniciar

- **DQ-008:** provider concreto (Vercel + Supabase + R2 vs Fly.io + Neon + R2). Decidir antes de la Fase 4.
- **DQ-011:** revisión legal sobre almacenamiento de datos contables paraguayos en cloud extranjera.
- **DQ-007:** ¿UI también en guaraní? Por defecto sólo español.
- **DQ-010:** modelo de cobro del SaaS al cliente final (V2).

Documentación complementaria en la carpeta *docs*/: SDD, modelo de datos, prompt Gemini, validaciones, análisis del Excel oficial, memoria viva del proyecto y log cronológico.